

Database Performance Audit — qdcerp / ERPReborn

Date: 2026-06-18 **Scope:** Application-side (PHP / Laravel) database interaction, connection configuration, runtime/caching, and serialization. Conducted by 5 parallel read-only investigators against the codebase. **Status:** Diagnosis only. **No fixes applied** — remediation will be delivered later.

Companion: the **server-side** (PostgreSQL stored-procedure) analysis already exists in project memory (`qdcerp-perf-findings` , `qdcerp-function-complexity-report.md`). That work established the dominant *DB-internal* cost is recursive camelCase JSON conversion (CPU) and the worst proc is the Advance picklist (~27s). **This audit covers the PHP/infrastructure side** and shows how the application layer *multiplies* those DB costs.

The headline: costs compound multiplicatively

Three independent layers each multiply the cost of every other:

per logical query → 5 DB round-trips

fetch a table → 10 round-trips (×2)

inside a loop → × N rows

unpaginated → whole table per request

(Layer 1: execution helper)

(Layer 1: DataOnly builder pattern)

(Layer 2: N+1 patterns)

(Layer 3: picklists/exports)

A 3N N+1 loop over 100 currencies (Finding N1.2), where each "call" is a DataOnly fetch, is on the order of **3 × 100 × 10 ≈ 3,000 DB round-trips** for one synchronize action — before counting the per-row×per-column PHP transform and the PHP-side camelCase re-walk on the result.

Severity matrix (worst first)

ID	Finding	Severity	Cost axis	File
Q1	<code>getQueryExecution()</code> = 5 DB round-trips/query across 2 unpooled connections	Critical	per-query	01-query-execution-layer.md
Q2	DataOnly fetchers call <code>getQueryExecution</code> twice → ~10 round-trips/fetch	Critical	per-request	01
C1	<code>enable_seqscan = off</code> set globally (planner sledgehammer)	Critical	server-wide	04-connection-config-runtime.md
Q3	SQL syntax validation runs a server-side stored proc on every query	High	per-query	01
Q4	Raw <code>pg_connect</code> / <code>pg_close</code> per query — no pooling/persistence	High	per-query	01 / 04
Q5	Full result set materialized + per-row×per-col PHP cast loop	High	per-row×col	01
S1	PHP-side recursive camelCase re-walk (twin of the DB cost, paid 2×)	High	per-cell	05-serialization-transformation.md
U1	~15 unbounded picklists load whole reference tables on form load	High	per-form	03-unbounded-fetches-pagination.md
U2	73/81 picklist blades render full result sets client-side (no <code>serverSide</code>)	High	per-form	03
U3	40 ExportExcel classes materialize whole dataset ~2× in PHP memory	High	per-export	03
N1	6 confirmed N+1 loops (currency sync worst at 3N+, nested)	High	per-row	02-n-plus-one-patterns.md
C3	No connection pooler (PgBouncer/Pgpool); <code>max_connections=100</code>	High	concurrency	04
C4	No <code>statement_timeout</code> — runaway query (27s picklist) holds a backend	High	concurrency	04
C5	Stock <code>work_mem</code> (4MB) / <code>shared_buffers</code> (128MB) in compose config	High	per-query	04
C6	Default DB driver is <code>mysql</code> not <code>pgsql</code> in 2 app variants	High	correctness	04
S2	<code>getArrayKeyRename_Engine</code> uses <code>create_function()</code> (removed in PHP 8)	Medium	per-call	05
Q6	Duplicate fetch impls + redundant full-array copy	Medium	per-request	01
S3	Per-query PHP datetime-string parsing for timing bookkeeping	Medium	per-query	05
C9	Scheduler <code>everyMinute</code> with no <code>withoutOverlapping</code> / <code>runInBackground</code>	Medium	periodic	04
U4	<code>getQueryExecutionDataFetch_DataOnly_All()</code> = <code>SELECT*/no-LIMIT</code> (dead code)	Low	latent	03

Cross-cutting themes (where fixes will have the most leverage)

- The execution helper is the single highest-leverage target.** Removing the `SELECT 1` probe, the server-side syntax-validation SP, and the two `SELECT NOW()` clock reads eliminates ~3 of every 5 round-trips on every query in the system, system-wide, with no behavior change. (Q1, Q3 — `Helper_PostgreSQL.php:909-1054`.)
- Connection churn.** `pg_connect` / `pg_close` per query (Q4) + no pooler (C3) means every query pays a Postgres backend-fork. A persistent/pooled connection plus PgBouncer neutralizes most of it.
- Pagination is the live unbounded-fetch problem**, not `DataOnly_All` (which is dead code). The gateway picklist endpoints with `'paging' => null` feeding client-side DataTables (U1, U2) are what actually drag the Advance flow to ~27s.
- The camelCase cost is paid twice** — once in the DB function (known), once again in PHP (S1).

Files in this folder

File	Dimension	Investigator
01-query-execution-layer.md	Core Helper_PostgreSQL execution path	A1
02-n-plus-one-patterns.md	DB calls inside loops	A2
03-unbounded-fetches-pagination.md	Picklists, exports, missing pagination	A3
04-connection-config-runtime.md	Connection config, pooling, cache, PG server	A4
05-serialization-transformation.md	camelCase / JSON / logging CPU overhead	A5

Methodology note: line numbers were reported by the investigating agents from the current `master` checkout. Re-verify exact lines before editing, as the helper files are large and actively changed.

01 — Core Query Execution Layer

Investigator: A1 (corroborated by A5) **Primary file:** `Programming/.LaravelCore/app/Helpers/ZhtHelper/Database/Helper_PostgreSQL.php` **Twin:** `Helper_ODBC.php`, `Helper_SQLValidation.php`

This is the centralized helper every model method routes through to call a PostgreSQL stored procedure (`getQueryExecution` has **294 call sites** across `app/`). Because every DB access funnels through it, overhead here is multiplied across the entire system.

Q1 — `getQueryExecution()` issues 5 DB round-trips per logical query, across two connections — CRITICAL

`Helper_PostgreSQL.php:909-1054`

A single `getQueryExecution($sql)` performs this sequence:

1. `getStatusAvailability()` → `DB::select('SELECT 1;')` — Laravel PDO connection (line 932 → 828)
2. `isValid_SQLSyntax()` → runs server-side SP `SchSysConfig.FuncSys_General_IsValid_SQLSyntax` — Laravel PDO (line 937 → 1671) (see Q3)
3. `SELECT NOW();` to capture start time — Laravel PDO (lines 947-951)
4. **the actual business query** — native `pg_connect` connection (lines 971-975)
5. `SELECT NOW() ... ::interval` to capture finish time — Laravel PDO (lines 984-996)

Steps 1, 3, 5 are pure overhead. The two `NOW()` clock reads (3, 5) run on a *different* connection than the real query (4), so the captured timing is both wasteful and inaccurate. Worse: lines 961-965 and 1007-1010 then *overwrite* those SQL clock values with PHP `new \DateTime()` for the actual TZ string — so round-trips 3 and 5 are almost entirely wasted work.

Cost: per-query. ~4ms of pure latency injected before any real work at ~1ms RTT; 5× connection/parse pressure under load. Multiplied across 294 call sites × queries-per-request.

Highest-leverage fix target (dropping steps 1, 3, 5 and Q3 removes ~3-4 of every 5 round-trips with no behavior change).

Q2 — "DataOnly" fetchers double everything → ~10 round-trips per fetch — CRITICAL

`Helper_PostgreSQL.php:1152-1191 (_All)`, `1213-1278 (_Filtered)`, `1279-1309 (_Specific)`

Each public fetcher calls `getQueryExecution()` **twice**: first to run a stored proc that *returns a SQL string* (a "QueryBuilderString"), then again to execute that returned string.

```
$varData = self::getQueryExecution($varUserSession, $varSQL); // 5 round-trips
$varSQL  = $varData['data'][0]['QueryBuilderString'];
$varData = self::getQueryExecution($varUserSession, $varSQL); // 5 round-trips
```

One logical "fetch table/record" ≈ **10 DB round-trips**. The query-builder SP runs server-side every time and the generated SQL is never cached.

Cost: per-request. This is the primary CRUD read path and the compounding multiplier behind slow list/picklist endpoints (and it is what compounds the N+1 loops in `02-n-plus-one-patterns.md`).

Q3 — SQL syntax validation runs a server-side stored proc on every query — HIGH

`Helper_PostgreSQL.php:1664-1691 (isValid_SQLSyntax)`, invoked at line 937

Validation is not a local regex — it is a full DB call into a PL/pgSQL function executed before *every* real query. The query text is serialized + single-quote-escaped in PHP (`getBuildStringLiteral_StoredProcedure` → `getStringLiteralConvertForVarChar`), shipped to the DB, parsed by the SP, and shipped back, purely to gate execution.

Cost: per-query — round-trip + server CPU, scaling with SQL text length.

Q4 — Native `pgsql` connection opened and closed per query (no pooling/persistence) — HIGH

`Helper_PostgreSQL.php:209 (pg_connect)`, `292 (pg_close)`, in `getArrayFromQueryExecutionDataFetch_UsingPGSQLConnection()`

`pg_connect` (not `pg_pconnect`) is opened fresh on every call and `pg_close` d at the end. The connection string is rebuilt from `DB::getConfig()` each call (lines 194-203). This is a *separate* connection from the Laravel PDO used in Q1 steps 1/2/3/5 — so **two independent connections per query**, neither pooled.

Cost: per-query connection establishment — a Postgres backend-fork is one of its most expensive ops; 2× `max_connections` pressure under concurrency. See also `04 C3/C4` (no pooler, no `statement_timeout`).

Q5 — Full result set materialized into PHP with a per-row × per-column transform loop — HIGH

`Helper_PostgreSQL.php:240-291`

After `pg_query` : builds a field-type map per column (`pg_field_type` , 240-245), then `while (pg_fetch_assoc)` loads **every** row into `$varData` (no LIMIT/streaming/cursor), and for **each** row runs an inner `foreach` over **every** column with a `switch` casting bool/int/float — calling `getBooleanConversion()` per bool cell (247-291). `getBooleanConversion()` (line 337) is itself wrapped in the full logger scaffolding, once per boolean cell.

Cost: O(rows × columns) PHP work + whole set held in memory. The dominant CPU/memory cost on wide/large results — consistent with the prior memory note that per-cell PHP transformation, not missing indexes, is the real hotspot.

Q6 — Duplicate fetch implementations + redundant array copy — MEDIUM

`Helper_PostgreSQL.php:114–167` (Laravel variant), `187–318` (PGSQL variant); twin `Helper_ODBC.php:90+`

`..._UsingLaravelConnection` (114) re-loops the result to cast each row `(array)$row` (140-144) — a second full pass over data already fetched by `getQueryExecutionDataFetch` (1073). `Helper_ODBC::getQueryExecution` (90) duplicates the same `NOW()`/loop pattern and has a `do { } while (executionTime < 0)` timestamp busy-guard (107-112).

Cost: per-request (extra full-array copy / redundant pass); plus maintenance divergence across the two implementations.

Q8 — Per-parameter SQL literal building done in PHP for every SP call — LOW/MEDIUM

`Helper_PostgreSQL.php:382+` (`getBuildStringLiteral_StoredProcedure`), `1578–1609` (`VarChar`), `1377+` (`BigInteger`)

For every SP invocation (including the validation SP of Q3), params are individually type-switched and concatenated with `explode / str_replace / substr / strcmp / in_array / strtoupper` per param (426-469), each `getStringLiteralConvert*` again wrapped in logger scaffolding.

Cost: per-query, scaling with param count + value length.

Priority within this layer

#	Title	Severity	Cost axis
Q1	5 round-trips/query across 2 connections	Critical	per-query
Q2	DataOnly fetchers double everything (~10 RT)	Critical	per-request
Q3	Syntax validation = server-side SP every query	High	per-query
Q4	pg_connect/pg_close per query, no pooling	High	per-query (connect)
Q5	Full materialization + per-cell transform loop	High	per-row×col
Q6	Duplicate fetch impls + redundant copy	Medium	per-request
Q8	Per-param PHP literal building	Low/Medium	per-query

Highest leverage: Q1+Q3 (drop the probe, the two clock reads, and the syntax SP — removes ~3-4 of every 5 round-trips, cheap to do). Q2 (cache the query-builder string) and Q4 (persistent/pooled single connection) are the biggest structural wins. Q5 is the dominant CPU cost on large reads.

02 — N+1 Patterns (DB calls inside loops)

Investigator: A2 **Scope searched:** Programming/.LaravelCore/app/**, Controllers, WebBackEnd/app, WebFrontEnd controllers.

Multiplier warning. Every "DB call" below routes through `getQueryExecution()` and is therefore **~5 DB round-trips** (or **~10** if it is a `DataOnly` fetch — see 01 Q1/Q2). So a **3N** loop over 100 rows is on the order of **1,500–3,000 actual round-trips**. The N+1 multiplier and the per-query overhead compound.

6 confirmed N+1 loops, worst first.

N1.1 — Currency exchange-rate sync (Central Bank) — HIGH (worst offender)

.../System/Transaction/Engines/synchronize/master/setCurrencyExchangeRateCentralBank/v1/setCurrencyExchangeRateCentralBank.php:129–156 and 207–219

```
for ($i = 0; $i != count($varDataOnline['data']); $i++) {
    if (!(new ...TblCurrency())->getCurrencyIDByISOCode($varUserSession, $...['ISOCode'])) { // call 1
        (new ...TblCurrency())->setDataInsert(...); // call 2
    }
    (new ...TblCurrencyExchangeRateCentralBank())->setDataImport(...); // call 3
}
// second block 190–221: NESTED — outer over offline files, inner over rows-per-file
for ($i = 0; $i != count($varOfflineFileList); $i++) { // ~10–20 files
    for ($j = 0; $j != count($varData['data']); $j++) { // ~25 rows each
        (new ...TblCurrencyExchangeRateCentralBank())->setDataImport(...);
    }
}
```

2–3 DB calls per outer iteration in the first loop, then a nested file×row loop (190–221). Multiplier **3N+** for the first loop and **N×M** for the nested block (could be 250+ inserts if all offline files are processed). This is the heaviest N+1 — and at ~5 round-trips per `setDataImport` call (01 Q1), the nested block alone can be ~1,250+ actual round-trips.

N1.2 — Currency exchange-rate sync (Tax) — HIGH

.../synchronize/master/setCurrencyExchangeRateTax/v1/setCurrencyExchangeRateTax.php:136–148

```
foreach ($varData['data']['exchangeRate'] as $varISOCode => $varExchangeRate) {
    (new ...TblCurrencyExchangeRateTax())->setDataImport($varUserSession, $varCurrentDate, ...);
}
```

One DB insert per currency. Multiplier **N**.

N1.3 — Login: per-branch roles + privileges — HIGH (hot path: every login)

.../Authentication/Engines/general/setLogin/v1/setLogin.php:599–636 (getOptionList)

```
for ($i = 0; $i != count($varData); $i++) {
    $varDataUserRole = (new ...SchSysConfig\General())->getDataList_UserRole($varUserID, $varData[$i]['Sys_ID']); // call 1
    $varReturnUserRole = (new ...SchSysConfig\General())->getUserRolePrivilege($varUserSession, $varUserID, $varData[$i]['Sys_ID']); // call 2
}
```

2 DB calls per branch. Multiplier **2N**. On the authentication path, so every login pays it.

N1.4 — Login: per-role menu layout — HIGH (hot path: every login)

.../setLogin/v1/setLogin.php:477–497

```
foreach ($varDataRole as $varDataRoles) {
    if (Redis::get("RedisGetMenu" . $user_RefID) == null) {
        $varMenu = (new ...SchSysConfig\General())->getUserPrivilege_MenuLayout($varUserSession, $varDataBranch[0]['Sys_ID'], $user_RefID);
        // ... Redis cache set
    }
}
```

One DB call per role (Redis cache check can skip some after first). Multiplier up to **N**. Note: the Redis key does not vary by role inside the loop, so the cache may mask the per-role intent — worth a correctness look during the fix.

N1.5 — Business document filter by document type — MEDIUM/HIGH

.../System/Report/Engines/form/resume/master/getBusinessDocumentFilterByDocumentTypeID/v1/getBusinessDocumentFilterByDocumentTypeID.php:73
84

```
for ($i = 0; $i < count($varBufferDB); $i++) {
    $varReturn = (new ...\SchData_OLTP_Master\General())
        ->getIDTranslation_BusinessDocumentVersionToBusinessDocumentForm(
            $varUserSession, $branchID, $varBufferDB[$i]['BusinessDocumentVersion_RefID']);
}
```

One DB call per buffered row (an early `break` mitigates best case). Multiplier **N** worst case — a per-row RefID→Form translation that is a classic batch/JOIN candidate.

N1.6 — Device person-access log sync — MEDIUM

.../System/DataWarehouse/Engines/synchronize/acquisition/setLog_Device_PersonAccess/v1/setLog_Device_PersonAccess.php:299–318

```
for ($i = 0; $i != count($varData); $i++) {
    if (/* time comparison */) {
        (new ...\SchData_Warehouse_Acquisition\TblLog_Device_PersonAccess())->setDataInsert(
            $varUserSession, null, substr($varData[$i]['dateTimeTZ'], 0, 4), 11000000000003,
            $varLog_Device_PersonAccessFetch_RefID, $varData[$i]['dateTimeTZ'], $varData[$i]['ID'], null);
    }
}
```

One insert per attendance-log entry. Multiplier up to **N**. Bulk-insert candidate.

Summary

#	Location	Multiplier	Path
N1.1	setCurrencyExchangeRateCentralBank	3N+ (nested)	sync
N1.2	setCurrencyExchangeRateTax	N	sync
N1.3	setLogin getOptionList	2N	login (hot)
N1.4	setLogin menu layout	N	login (hot)
N1.5	getBusinessDocumentFilterByDocumentTypeID	N	report
N1.6	setLog_Device_PersonAccess	N	sync

Pattern: all are per-row inserts/lookups that are batch- or set-operation candidates (bulk insert, single JOIN, or a set-based stored proc). The currency sync (N1.1) and the two login loops (N1.3/N1.4) are the priorities — N1.1 for raw volume, the login pair because they sit on the most-traveled path.

Ruled out (checked, NOT N+1) — do not spend fix effort here

The codebase deliberately pushes joins into stored procedures, so genuine N+1 is rare and concentrated. These were investigated and cleared — they bulk-insert or assemble arrays in memory rather than calling the DB per row:

- `setAdvancePayment` — builds payload in memory / bulk path.
- `setFilesAppend` — in-memory array assembly, not per-row DB calls.
- Combined-budget builder — array construction in memory.

Note: A2's scan focused on the `LaravelCore` backend engines. The picklist/export hotspots (`03-unbounded-fetches-pagination.md`) are a different shape (one big unbounded query rather than per-row loops) and are tracked separately.

03 — Unbounded Fetches, Picklists & Pagination

Investigator: A3 **Scope:** picklist/dropdown loaders, `getQueryExecutionDataFetch_DataOnly_All` callers, ExportExcel controllers, `getFunction` blades.

Headline: the *live* unbounded-fetch problem is the **gateway picklist endpoints with 'paging' => null feeding client-side DataTables** — not the `DataOnly_All` helper (which is dead code). **73 of 81** `getFunction` picklist blades render the full result set client-side; only 2 use `serverSide:true`. This is the PHP/UI side of the ~27s Advance picklist.

U1 — ~15 unbounded picklist/dropdown loaders fetch whole reference tables on form load — HIGH

All follow the same pattern: call `Helper_APICall::setCallAPIGateway()` with `'paging' => null` (or no paging params), then hand **all** rows to the frontend. Top offenders (largest tables × most frequent):

#	Loader	Proc	File:line
1	Chart of Accounts	<code>dataPickList.accounting.getChartOfAccount</code>	<code>WebFrontEnd/app/Http/Controllers/Function/FunctionController.php:376</code>
2	Workers / Requester / Beneficiary	<code>transaction.read.dataList.humanResource.getWorkerJobsPositionCurrent</code>	<code>FunctionController.php:229-252, 282-316, 318-352</code>
3	Warehouses / Deliver-To	<code>transaction.read.dataList.supplyChain.getWarehouse</code>	<code>FunctionController.php:420-451, 454-486</code>
4	Purchase Requisitions	<code>dataPickList.supplyChain.getPurchaseRequisition</code>	<code>Services/Purchase/PurchaseRequisitionService.php:13-15</code>
5	Suppliers	<code>dataPickList.supplyChain.getSupplier</code>	<code>Services/Master/Supplier/SupplierService.php:36-61</code>
6	Transporters	<code>dataPickList.supplyChain.getTransporter</code>	<code>Services/Master/Transporter/TransporterService.php:36-61</code>
7	Material Receive orders	<code>dataPickList.supplyChain.getWarehouseInboundOrder</code>	<code>Services/Inventory/MaterialReceiveService.php:77-92</code>
8	Material Return orders	<code>dataPickList.supplyChain.getWarehouseOutboundOrder</code>	<code>Services/Inventory/MaterialReturnService.php:35-49</code>
9	Payment Instructions	<code>dataPickList.finance.getPaymentInstruction</code>	<code>Services/Finance/AccountPayableService.php:65-79</code>
10	Advance Requests	<code>dataPickList.finance.getAdvance</code>	<code>Services/Process/Advance/AdvanceRequestService.php:13-15</code>
11	Advance Settlements	<code>dataPickList.finance.getAdvanceSettlement</code>	<code>Services/Process/Advance/AdvanceSettlementService.php:13-15</code>
12	Business Trip Requests	<code>dataPickList.humanResource.getPersonBusinessTrip</code>	<code>Services/Process/BusinessTrip/BusinessTripService.php:13-15</code>
13	Debit Notes	<code>dataPickList.finance.getDebitNote</code>	<code>Services/Process/DebitNote/DebitNoteService.php:12-13</code>
14	Journal Entries	<code>dataPickList.accounting.getJournal</code>	<code>Services/Accounting/JournalService.php:13-27</code>
15	Sales Contracts	<code>dataPickList.customerRelation.getSalesContract</code>	<code>Services/Sales/CustomerOrderService.php:13-27</code>

These load **synchronously on form page load** (`$(window).one('load', ...)`), blocking the UI until the entire table arrives. The fetched payload then pays the per-cell PHP transform (`01 Q5`) and camelCase re-walk (`05 S1`).

U2 — 73/81 picklist blades render full result set client-side (no server-side pagination) — HIGH

Across `Programming/WebFrontEnd/resources/views/getFunction/` only 2 blades use `DataTables serverSide:true`; the other 73 use `data: data` (full client-side load).

The Advance flow is the worst concrete instance:

- `AdvanceRequestService::getPickList()` → gateway `dataPickList.finance.getAdvance (parameter => null, NO paging)` — the ~27s stored proc.
- `WebFrontEnd/app/Http/Controllers/Process/Advance/AdvanceRequestController.php:177-206` (`AdvancePickList`) fetches **all** rows, applies only client-side `array_filter()` on `project_id/site_id` (lines 191-200), returns the whole array.
- Blade `resources/views/getFunction/getAdvance.blade.php:63` loads it into `DataTables` with `deferRender/scroller` (virtual scrolling can't help — all rows are already loaded from the DB).
- Same pattern for `AdvanceSettlementController.php:184-199` (`AdvanceSettlementPickList`).

Contrast — these are done correctly (paginated): the *Summary report* endpoints `ReportAdvanceSummaryStore` (`AdvanceRequestController.php:227–281`, server-side `length / start` → SP paging) and `ReportAdvanceToASFStore` (`:541–607`) use `serverSide:true`. So the codebase already has the correct pattern to copy. The unpaginated one to note is `ReportAdvanceSettlementSummaryStore` (`AdvanceSettlementController.php:372–409`) — fetches all rows.

U3 — 40 ExportExcel classes materialize the whole dataset ~2x in PHP memory — HIGH

Programming/WebFrontEnd/app/Http/Controllers/ExportExcel/ (40 classes)**

All implement `Maatwebsite FromCollection + FromArray` (in-memory); **none** use `FromQuery` (the chunked/streaming path). Each export holds the full result set in memory, then builds a **second** full copy while formatting rows.

- Representative: `ExportExcel/AdvanceRequest/ExportReportAdvanceSummary.php:28` `collection()` loops the entire dataset into `$filteredData[]` (a full duplicate) before `collect()`.
- Data path: `Process/Advance/AdvanceRequestController.php:283` (`PrintExportReportAdvanceSummary`) defensively raises `ini_set('memory_limit','512M') + set_time_limit(180)` (lines 286–287) and gets its dataset from `json_decode($request->dataReport)` (line 296) — the **entire report is round-tripped from the browser as a POST body**, then re-materialized for Excel. The 512M/180s bumps are a tell the payloads are already large and growing.
- Confirmed app-wide, not Advance-specific: same pattern in `ExportExcel/Inventory/ExportReportMaterialReceiveSummary.php:28`.

Cost: ~2x dataset in memory per export, unbounded by row count, no server-side chunking/streaming. Concurrent exports can exhaust PHP memory under a constrained `memory_limit`.

U4 — `getQueryExecutionDataFetch_DataOnly_All()` is SELECT-*/no-WHERE/no-LIMIT, but dead code — LOW (latent)

Helper_PostgreSQL.php:1152 builds a column list via SP `SchSysConfig.FuncSys_General_GetStringLiteralFieldSelect_DataOnly_All(...)` then runs the generated SELECT with **no WHERE and no LIMIT** — a full-table fetch of every authorized row/version.

- Only PHP caller: `Models/BackEnd/Database/DefaultClassPrototype.php:68` (`getAllDataRecord()`).
- **Blast radius:** `getAllDataRecord()` has **zero** callers in `WebFrontEnd`; in `.LaravelCore` it appears only in `framework/sandbox/cache/CloudStorage` scaffolding, not request paths.
- Sibling `getQueryExecutionDataFetch_DataOnly_Filtered()` (`:1213`) has its execution commented out (`// $varData = ...`) and an active `echo $varSQL debug line (:1246)` — broken/unused.

Severity LOW in practice — a latent landmine if it ever gets wired into a request path, but it is not a live hot path today. The real unbounded fetches are U1/U2.

Priority within this dimension

1. **U2 / U1 — picklists.** Push pagination into the gateway `dataPickList.*` calls and switch the 73 client-side blades to `serverSide:true` (the Summary reports already show how). This is what makes the Advance flow feel ~27s.
2. **U3 — exports.** Move to `FromQuery` / chunked streaming; stop round-tripping the full report through the browser POST body.
3. **U4** — leave for cleanup; remove or guard the dead `DataOnly_*` helpers.

04 — Connection Config, Runtime/Caching & PostgreSQL Server

Investigator: A4 **Scope:** `config/database.php`, `cache/session/queue` config, Docker `postgresql.conf` / `pg_hba.conf`, scheduler, ODBC.

CRITICAL

C1 — `enable_seqscan = off` set globally

`.ProjectCore/Configuration/Docker/PostgreSQL/System/var/Lib/postgresql/data/postgresql.conf` → `enable_seqscan = off` (active). (The PG18 `.../18/docker/postgresql.conf` actually mounted by compose leaves it at default `on` — which file is live depends on which config the running server loads.)

A planner sledgehammer: forces the optimizer to avoid seq scans even when cheapest (small tables, high-selectivity), producing pathological plans — index scans that touch every heap row, absurd nested loops. Across 6 federated DBs this makes cheap queries expensive. Should be `on`; fix specific plans with indexes/ `ANALYZE`, not this global switch.

HIGH

C3 — No connection pooler (PgBouncer/Pgpool) anywhere

No `pgbouncer` / `pgpool` in `.ProjectCore` or compose. `max_connections = 100` in both `postgresql.conf` files. Every request/query bears connection-establishment cost; combined with the per-query `pg_connect` / `pg_close` churn (01 Q4 / C-below) and Apache per-process connections, connection storms can exhaust the 100 cap ("too many clients") and queue. A transaction-pooling PgBouncer in front of .27 would neutralize most of the churn.

C4 — No `statement_timeout` (or `connect_timeout` / `application_name`) in any pgsq block

The `config/database.php` pgsq blocks (WebBackend ~70-80) define only `host/port/db/user/pass/charset/sslmode` — no `options`, no `PDO::ATTR_PERSISTENT`.

- No `statement_timeout` → a runaway query (e.g. the ~27s Advance picklist) holds a backend + connection indefinitely, starving the 100-cap pool.
- No `application_name` → can't attribute load in `pg_stat_activity`.
- `sslmode=prefer` adds a TLS-negotiation attempt per new connection (compounds connection churn).

C5 — Stock `work_mem` / `shared_buffers` in the compose-mounted config

`.../18/docker/postgresql.conf` → `shared_buffers = 128MB` (line 132), `work_mem` commented = **4MB** default (line 143), `effective_cache_size` commented = 4GB default. (The `data/` variant raises these to 4GB/64MB/16GB — appropriate, but possibly not the config the running server loads.) 4MB `work_mem` forces large sorts/hash-joins/aggregations to **spill to disk temp files** — directly slows the report/picklist queries. 128MB `shared_buffers` caches almost nothing.

C6 — Default DB connection is `mysql`, not `pgsql`

`WebBackend/config/database.php:19` and `WebFrontEnd/config/database.php:19` → `'default' => env('DB_CONNECTION', 'mysql')`. `WebFrontEnd/.env:11` is still `DB_CONNECTION=mysql`, `DB_HOST=127.0.0.1`, `DB_DATABASE=laravel` (stale scaffold). Anything using the framework default (DB-backed `queue/cache/session`, ad-hoc `DB::` calls) without an explicit connection hits the wrong driver; WebFrontEnd's pgsq block has no real target.

MEDIUM

C9 — Scheduler `everyMinute` with no `withoutOverlapping` / `runInBackground`

`WebBackend/app/Console/Kernel.php:29` (`ScheduledTask_EveryMinute->everyMinute()`); `grep withoutOverlapping|runInBackground` → **0 matches**. Cron `* * * * * artisan schedule:run`; plus a `SwingBarrierGate` SCP import `*/15 * * * *`. Without `withoutOverlapping`, a >60s task stacks into concurrent DB-hammering runs; without `runInBackground`, all due tasks run serially holding DB connections. The every-minute task opens a DB-backed user session each tick (`ScheduledTask_EveryMinute.php:42`).

LOW / NOTES

C11 — `pg_hba.conf` auth

`.../data/pg_hba.conf`: `local all all trust, remote host all all all md5`. `md5` (not `scram-sha-256`) is weaker; the auth round-trip is part of the per-connection cost. Mostly a security note.

C12 — `dblink` is not in app code

`grep dblink` over `WebBackend/app` + the PG helper → 0 matches. The 6-DB federation lives entirely in Postgres stored procedures, not PHP. Each server-side `dblink` opens its own connection to the target DB — **not poolable from the app layer** (relevant context for any consolidation discussion).

Highest-leverage fixes (connection/runtime)

1. Replace per-query `pg_connect / pg_close` (`Helper_PostgreSQL.php:209/291`) with a reused/persistent connection — biggest churn source (ties to `01 Q4`).
2. Set appropriate `work_mem / shared_buffers` on the running server and ensure `enable_seqscan=on` (C1/C5); the mounted PG18 conf is all defaults (4MB `work_mem`, 128MB `shared_buffers`).
3. Add PgBouncer (C3); set `statement_timeout + application_name` in the `pgsql` blocks (C4).

05 — Serialization & Transformation (CPU overhead)

Investigator: A5 **Scope:** `Helper_Array`, `Helper_JSON`, `Helper_String`, `Helper_DateTime`, `Helper_API`, `Helper_SystemLog`.

The prior DB-side analysis found the single dominant *DB-internal* cost is recursive camelCase JSON key conversion (CPU-bound, recursive dynamic SQL in PG functions). This dimension finds the **PHP-side twin** (the same work paid a second time) plus the per-query timing CPU. It also **clears** the logging scaffolding as a production hotspot.

S1 — PHP-side recursive camelCase key rename — the exact twin of the DB cost, paid 2x — HIGH

`Helper_Array.php:102-134` (`getArrayKeyRename_CamelCase`) **Entry:** `Helper_API.php:397-440` (`getEngineDataSend_DataRead`), default arg `$varSignConvertPHPArrayToJSONCamelCase = TRUE`.

`getArrayKeyRename_CamelCase` recurses the entire result array and, for every key, does up to 4 `substr`

- 2 `strtoupper` + `strcmp` (lines 113-119), rebuilding a brand-new array. This is the PHP equivalent of the recursive dynamic-SQL camelCase already running in Postgres — so **the same transformation is paid twice**: once in the DB function, again here. `getEngineDataSend_DataRead` is called in **624 controller files**, so essentially every list/read endpoint re-walks and re-allocates its full result set key-by-key.

Cost: scales with payload size (rows × columns × nesting depth) — a per-cell CPU tax on top of the DB cost. Worst on the large unbounded picklists (`03 U1/U2`).

S2 — `getArrayKeyRename_Engine` uses deprecated `create_function()` — MEDIUM (also a PHP 8 fatal)

`Helper_Array.php:240-323` (Lower/Upper/LowerFirst variants route through it at 152-222)

`getArrayKeyRename_Engine` calls `create_function()` (lines 262, 282-309) to build the rename closure on **every** invocation. `create_function` is eval-based (recompiles a function each call) **and was removed in PHP 8.0** — so on PHP 8 these paths fatal; on PHP 7 they pay an eval+compile cost per call. Same recursive full-array rebuild as S1 (line 316). Lower-frequency than the camelCase path but the same O(cells) shape — and a correctness landmine for any PHP 8 runtime.

S3 — Per-query PHP datetime-string parsing for execution-interval bookkeeping — MEDIUM

`Helper_DateTime.php:797+` (`getDifferenceOfDateTimeTZString`), called from `Helper_PostgreSQL.php:1022-1027`

For every `getQueryExecution`, this runs a chain of `str_replace` + multiple `explode` + nested try/catch float casts + `getUnixTime` calls (lines 808-878) just to format `process.DBMS.executionTime.interval`. It does **not** hit the DB (good), but it is non-trivial pure-PHP string work executed once per query (× 294 call sites × queries-per-request) to produce a timing field most callers never read.

Cost: fixed per-query CPU, independent of payload. Pairs with `01 Q1` — the two `SELECT NOW()` round-trips feed this, and the result is mostly unused.

S-note — Logging/profiler scaffolding is NOT a production hotspot (cleared)

`Helper_SystemLog.php`, `Helper_QueryProfiler.php`

The original suspicion was that the `setLogOutputMethodHeader/ProcessHeader/Footer` wrappers on every helper call do heavy session string-building. **They don't, in production:** `Helper_SystemLog` short-circuits via `isTraceEnabled()` (memoized `env('SYSLOG_TRACE_ENABLED', false)`, default OFF) at the top of every method (lines 140, 166, 182, 234, 246, 417, 449, 494, 535, 572) — cheap early returns. `Helper_QueryProfiler::record` is in-memory only, gated by `isEnabled()` (line 74), bounded buffer, no file/DB writes.

Action: verify `SYSLOG_TRACE_ENABLED=false` in the production `.env`. If it is ever set true, the scaffolding becomes a **Critical** throughput killer because of the call multiplication in `01 (Q1/Q2/Q5/Q8)` — thousands of session read/write cycles per request.

Bottom line for this dimension

The DB-side camelCase cost is **mirrored in PHP (S1)** and effectively paid twice. The other items (S2 deprecated `create_function`, S3 per-query datetime parsing) are smaller, fixed-per-call CPU taxes. The biggest *app-side* latency amplifier is **not** serialization but the round-trip instrumentation in `01 (Q1)` — serialization here is the CPU that scales with payload size, while Q1 is the latency that scales with queries-per-request. Both compound on the large unbounded picklist payloads.